

Project Final Report

FoodieFind

Team-members

- Ash Aung
- Amir Matianiu
- Damon Yiu
- Jim Chen

Github: <https://github.com/CMPT-276-SPRING-2025/final-project-12-woods.git>

FoodieFind: <https://foodiefindz.netlify.app/>

Video Demo: https://youtu.be/2rNCe_G50Q8

Table of contents

- [1. FoodieFind](#)
- [2. Project Success in Meeting User Needs and Solving Problems](#)
- [3. Analysis of the Agile SDLC Model and its Implementation](#)
- [4. Detailed API Features in FoodieFind](#)
- [5. CI/CD Pipeline](#)
- [6. FoodieFind's testing strategy](#)
- [7. Diagrams of the Project's Architecture](#)
- [8. List of Known Bugs/Issues, and Their Severity](#)
- [9. Future Work and Potential Improvements](#)
- [10. Lessons Learned and Project Takeaways](#)
- [11. Appendix](#)

Project Success in Meeting User Needs and Solving Problems

Users appreciated the interactivity of FoodieFind and found most features operational and user-friendly. The AI-powered meal planning feature was particularly popular, assisting users effectively in making dining decisions. Additionally, the user interface was praised for its visual appeal and overall ease of navigation.

Areas for Improvement:

User feedback highlighted a few issues, notably occasional inaccuracies with location data and broken redirection links. The ChatBox, although recognized as helpful, required improvements to avoid repetitive loops and to enhance its visual spacing. Furthermore, the Explore page's real reviews feature, although beneficial, encountered functionality problems. The plan page could display more accurate results, such as pricing and restaurants.

User Recommendations for Future Enhancements:

- Resolve technical issues related to location accuracy and link redirections.
- Improve the ChatBox experience by addressing looping interactions and enhancing visual spacing.
- Repair the functionality of the real reviews feature to ensure consistent reliability.

FoodieFind effectively meets user expectations by providing a valuable, intuitive, and interactive platform for restaurant discovery. Addressing identified technical and usability improvements will further enhance user satisfaction and project effectiveness.

Analysis of the Agile SDLC Model and its Implementation

For this project, we went with the Agile Software Development Life Cycle (SDLC) because it gave us more flexibility and helped us build things gradually. Since we were working on features like AI-based suggestions, interactive maps, and a meal planner, Agile's step-by-step approach made it easier to test and improve things along the way.

We chose Agile for a few reasons:

- It let us break the work into smaller parts, so we could focus on each piece without getting overwhelmed.
- We were able to take feedback from users and classmates and use it to adjust things like the layout, recommendations, and how people interacted with the app.
- Agile allowed us to change the plan when needed. If something new came up, we could shift without having to redo everything.
- Regular check-ins helped keep everyone on the same page—especially between the design and dev teams.
- The short sprints meant we could build, test, and improve features quickly, instead of waiting until the very end.

Challenges We Ran Into:

One of the first problems we had was missing the deadline for Milestone 1.5 because some features tied to the first API weren't finished. But thanks to Agile, we were able to adjust our timeline and catch up. After reshuffling tasks, we got everything running for both APIs. Peer testing helped a lot too—it helped us catch bugs and polish the experience.

Overall, using Agile made our process more organized, helped us respond to problems quickly, and made our FoodieFind app better with each round.

Detailed API Features in FoodieFind

1. Google Maps API

- User Location Services:
 - Automatic geolocation detection with fallback to default location
 - Custom location search with Google Places autocomplete
 - Visual radius indicator with adjustable search distance
- Restaurant Discovery:
 - Nearby places search filtered for food establishments
 - Restaurant-specific search with autocomplete
 - Rating-based marker visualization (different emoji indicators)
 - Information windows with restaurant details on click
- Interactive Features:
 - Custom pin dropping with emoji/text labels
 - Map navigation and zooming
 - Searched restaurant history tracking
- API Implementation Details:
 - Uses @react-google-maps/api React wrapper
 - Integrates Places and Geometry libraries
 - Handles geolocation errors gracefully

Changes made since the planning stage:

- Interactive Map:
 - Planning Stage: Users can explore restaurants via zooming, panning, and typing specific locations; map interactions lead to restaurant details.
 - Actual Implementation: Matches the initial proposal, with enhanced usability through automatic geolocation detection, visual search radius adjustments, and seamless Google Places autocomplete integration.
- Dropped Pins:
 - Planning Stage: Hover interactions displaying restaurant details (name, cuisine, rating, cost) with clickable pins linking to Google Maps.
 - Actual Implementation: Realized fully, including interactive markers with emoji-based rating indicators, information windows appearing on hover, and easy navigation to the detailed Google Maps view.

- Hidden Gems:
 - Planning Stage: Focused explicitly on highlighting underrated, highly-rated restaurants with fewer reviews.
 - Actual Implementation: Not explicitly differentiated in the final description provided, suggesting either integration into general discovery features or possibly omitted as a distinct labeled feature. The implemented version emphasizes rating visualization but does not clearly outline a separate hidden gems functionality.
- Color-Coding Restaurants (Popularity or Price):
 - Planning Stage: Proposed clear color-coded distinctions based on popularity (ratings translated into scores) or price, with a legend.
 - Actual Implementation: Evolved into rating-based emoji indicators rather than numerical scoring or explicit color-coding by price or popularity. The final implementation shows ratings visually through emojis, simplifying visual representation but not explicitly following the planned legend/color-coding structure

2. Gemini AI API

- AI Chatbot:
 - Conversational interface for food-related queries
 - Message history tracking and display
 - Markdown rendering for formatted responses
 - Loading states during API requests
- Meal Planner:
 - Personalized meal plan generation based on:
 - User-specified daily budget
 - Dietary restrictions/preferences
 - Location-specific restaurants
 - Structured output with:
 - Restaurant recommendations for breakfast, lunch, and dinner
 - Direct Google Maps links to restaurant locations
 - Website links when available
 - Dish recommendations with price estimates
 - Dietary restriction compatibility notes

Changes made since the planning stage:

- Chatbox Feature:

- Planning Stage: Intended as a conversational guide for indecisive users, recommending food, cuisines, and answering general food-related queries.
 - Actual Implementation: Fully realized as a conversational AI Chatbot that supports user interaction, maintains message history, formats responses using Markdown, and shows loading states clearly. Remains aligned closely with initial intentions.
- Food Recommendation:
 - Planning Stage: Proposed a structured input (location, cuisine, budget) resulting in three nearby restaurant recommendations.
 - Actual Implementation: Incorporated as part of the AI-driven Meal Planner feature rather than a standalone recommendation component. It dynamically considers location, dietary restrictions/preferences, and budgets, delivering structured meal suggestions rather than only three restaurant locations.
3. Plan Meals:
- a. Planning Stage: Intended to provide users with a full day's meal (breakfast, lunch, dinner), including locations and average costs, based on their budget, dietary restrictions, and location.
 - b. Actual Implementation: Fully implemented, closely matching original plans, offering personalized meal plans with clear details, direct map links, website information (if available), and dietary compatibility notes.

4. Firebase Realtime Database

- Restaurant Recommendations:
 - CRUD operations for community restaurant sharing:
 - Add restaurants with name, location, price, and Google Maps link
 - Display all shared restaurants in a grid layout
 - Like/unlike voting system with counters
 - Remove functionality for user's own submissions
 - Validation for submissions including Google Maps link verification
- Food Trend Sharing:
 - Community food discovery system:
 - Share dishes with descriptions and photos
 - Photo upload with base64 encoding
 - Cuisine categorization with standardized options
 - Grid display of all shared dishes
 - Remove functionality for dishes

CI/CD Pipeline

Continuous Integration (CI)

CI was implemented using GitHub Actions, which automatically triggered workflows whenever one of our team members pushed code or opened a pull request to the dev branch. This automation ensured continuous feedback on code quality and functionality.

Unit and Component Testing

Tests were written using Jest and React Testing Library to validate the functionality of core components such as the map interface, search bar, and Gemini API integrations. The CI pipeline ran all tests during each build and failed if any test did not pass.

Unit tests ensured that individual components (e.g., location markers are displayed correctly, restaurants are found) behaved as expected. Integration tests verified that APIs, such as Google Maps and Gemini, returned correct data and integrated smoothly with the frontend.

Continuous Deployment (CD)

Deployment was handled through Netlify, which offered seamless integration with GitHub and automated the deployment process. The pipeline was configured to deploy changes directly from the dev branch or a preview on a pull request to the dev branch. As soon as a team member pushed to this branch, Netlify initiated a new deployment.

This workflow allowed us to see real-time updates on a live environment, and since we relied on manual testing on features under development, this significantly reduced the time for testing and feedback.

Monitoring and Post-Deployment Checks

During development sprints, to monitor and validate new features and user interactions, we would test each other's feature implementations. After every deployment, our team manually verified that the app loaded correctly and that all integrated APIs responded as expected on the live website

FoodieFind's testing strategy

Testing Strategy and Implementation

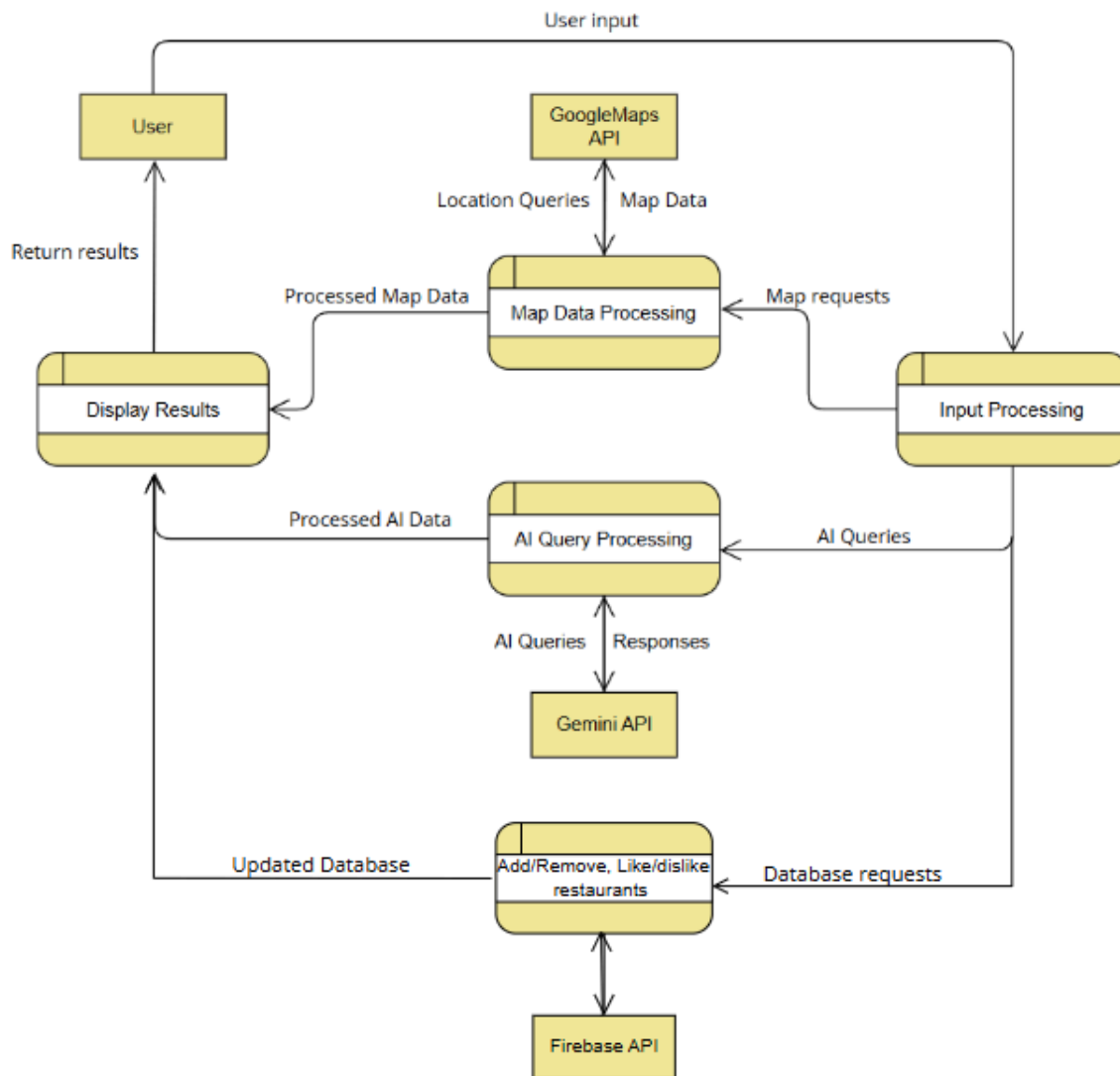
The FoodieFind project adopted a pragmatic testing strategy focused on maintaining reliability while allowing for fast and flexible development. While some automated tests were included, the team heavily relied on manual testing as the primary quality assurance method. This approach was particularly effective for testing features which were still in the experimental phase. Although manual testing was emphasized, our team also implemented some automated tests using Jest and React Testing Library. The automated tests ran through a GitHub Actions workflow, which were triggered every time code was pushed or a pull request was submitted to the dev branch. This ensured that basic automated checks were always performed, and builds would fail if any test did not pass.

Testing Objectives

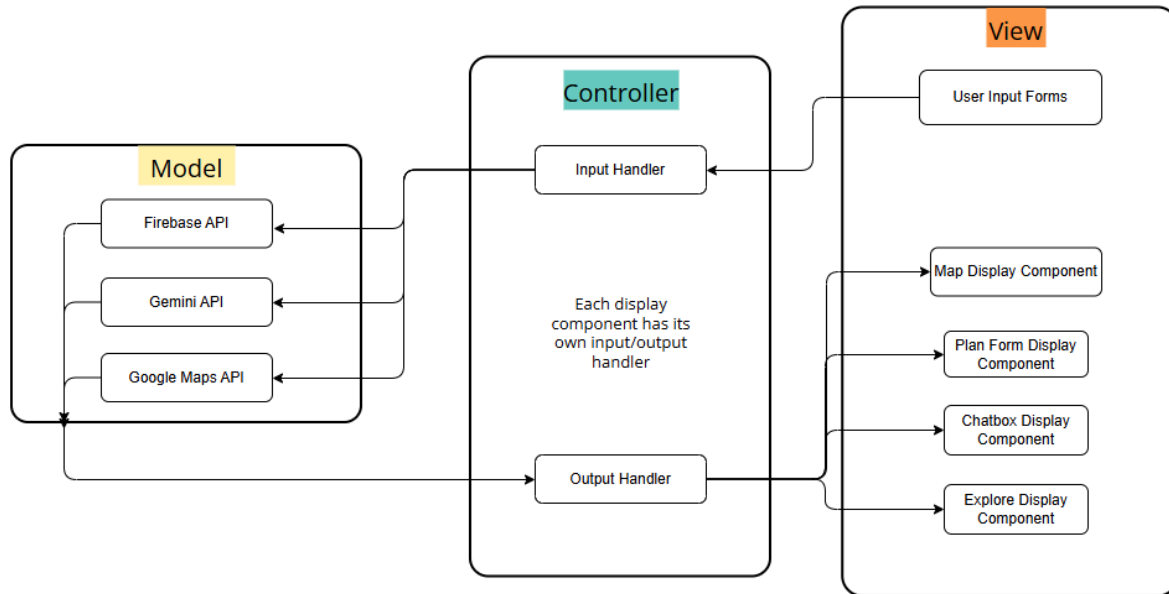
The main goals of the testing process were to identify and fix bugs in individual features before they were merged to the dev branch. Our team ensured consistent behavior across the user interface and verified integration with external APIs (such as Google Maps and Gemini). Each team member was responsible for testing their own features before marking them as complete. In addition, other team members tested each feature to identify bugs or issues from a fresh perspective. Any problems or inconsistencies were reported during weekly team meetings, where developers presented their progress and received feedback. This team-driven process helped catch bugs, usability concerns, and design inconsistencies that might have been missed by a single developer.

Diagrams of the Project's Architecture

DFD LVL 1 - Gane and Sarson diagram:



MVC - Model View Controller:



Flow Summary

1. The user interacts with **Input Forms** → Data is sent to the **Input Handler**.
2. Input Handler calls appropriate API(s) via the **Model**.
3. API responses are returned → **Output Handler** processes them.
4. Processed data is sent to the respective **Display Components** in the View.
5. Each component has its own logic for handling and rendering that data.

List of Known Bugs/Issues, and Their Severity

Medium Severity Issues

Bug/Issue	Description	Impact
AI Response Quality	No verification of restaurant existence in AI recommendations Steps to recreate: 1. Go to plan page 2. Input data into the input fields 3. Look through the generated plan 4. Click on the maps link and verify the existence of the restaurant 5. If everything is right, click on generate another plan again 6. Repeat steps 3-5 until found a non-existent restaurant https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/70	Users may receive recommendations for closed/non-existent restaurants
Link Format Consistency	Despite explicit instructions in the prompt, the Gemini AI often generates: - Short URLs (maps.app.goo.gl links) - Direct Google Maps URLs with place IDs - URLs with different parameter formats - Sometimes completely broken or malformed links Steps to recreate: 1. Go to plan page 2. Input data into the input fields 3. Look through the generated plan and click on the Maps URL 4. If Maps URL corresponds with the address, click generate plan again 5. Repeat steps 3-5 until found a URL that doesn't match the address or is broken https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/65	Links may malfunction or no links at all.

Issue with Restaurant Website Links and Addresses	The system has no validation mechanism to check if website URLs actually work. URLs generated by Gemini aren't verified before display. Gemini often generates addresses in various formats. Address formatting varies by country, and Gemini might use incorrect conventions Steps to recreate: 1. Go to plan page 2. Input data into the input fields 3. Look through the generated plan and look at the address 4. Click on the maps URL and see if that place pops up 5. If yes, click on generate plan again 6. Repeat steps 3-5 until the address of the restaurant is wrong. It doesn't show up as a place in the maps URL https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/66	Users might have to look up the restaurant address and the website manually.
Restaurant Pins placed outside the radius	When reducing the radius to a lower number (ex. From 3 km to 1km) the markers can sometimes show up outside the radius Steps to recreate: 1. Go to home page 2. Scroll down to the maps feature 3. Increase the radius to any number greater than 2 4. Decrease back to a lower number (eg. 1) https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/67	Visual inconsistency, but no impact on functionality
When switching locations on the map, need to update the radius as well	When switching to another location on the map. The user needs to additionally change the radius size in order to see the restaurant markers show up Steps to recreate: 1. Go to home page 2. Scroll down to the maps feature	User would have to update the radius manually each time they want to change their location

	3. Enter a different user location 4. Wait until places load (nothing will show up) 5. Increase the radius (Places will show up in the changed location) https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/68	
--	---	--

Low Severity Issues

Bug/Issue	Description	Impact
Invalid Location Handling	Insufficient validation for location inputs. Steps to recreate: 1. Go to Plan page 2. Enter a non existent place in the location input field 3. AI will try it's best to interpret the location to a real one https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/71	Users can enter nonsensical locations like "XYZ123" and the AI will attempt to generate recommendations.
Emoji Marker Inconsistency	Emojis render differently across operating systems and browsers. No fallback for systems with limited emoji support. Steps to recreate: 1. Open foodiefind's explore page on a mobile device 2. Open foodiefind's explore page on a laptop 3. On the mobile there will be flag emojis 4. On the laptop there will be text instead https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/62	There might be visual inconsistencies across platforms.
Google Maps API console: Uncaught type error	When switching pages, if the Google Maps API has not finished loaded the places, the console will throw an error; Steps to recreate: 1. Go to home page	An error will be shown in the console. No other impact is present to the user experience

	2. Open console window (ctrl+shift+i) 3. Scroll down to the maps feature 4. Before the places load in, switch to another page (eg. explore) 5. Look at the error message in the console window https://github.com/CMPT-276-SPRING-2025/final-project-12-woods/issues/63	
--	--	--

Future Work and Potential Improvements

Looking ahead, there are several areas we'd like to improve and expand on for FoodieFind. First, we want to address the existing bugs and inconsistencies we encountered during testing. This includes improving the chatbot and plan page to return more accurate and realistic results, since it occasionally recommends restaurants or menu items that don't exist, or lists prices that are too low to be realistic. For the map, we plan to make it more fluid and responsive, and we want to enhance it by displaying a photo of each restaurant alongside its location to give users a better visual experience. Additionally, remove the bugs due to location and radius change.

One feature we didn't have time to fully implement was a quick-recommendation tool, a simple page where users could answer a few short questions and instantly get 3 nearby restaurant suggestions. In the future, we'd like to combine the Gemini API for smart recommendations with the Google Maps API to show locations and photos of matching restaurants.

Lastly, we'd like to improve performance overall by refining the refresh and data-loading mechanisms, as we noticed some stuttering during interactions. These improvements would help make FoodieFind more seamless, accurate, and user-friendly.

Lessons Learned and Project Takeaways

During the development of FoodieFind, we faced several technical and teamwork-related challenges that helped us grow as developers. In the beginning, we weren't sure how to start, the project felt overwhelming, and since we didn't know each other well yet, it was hard to collaborate effectively. Keeping up with deadlines and communicating clearly was also difficult at first, especially as we prepared for user testing. We faced technical challenges like building a chatbot that could remember previous messages and respond accurately. We also had trouble with the Google Maps API, especially making the map feel smooth, customizing markers, and dealing with inconsistent data from the Places service. We ran into merge conflicts on GitHub as we worked on different features at the same time. Moreover, we dealt with a fair share of real-life conflicts.

We learned how to break down large problems into manageable tasks and prioritize work effectively. Adopting an Agile workflow with weekly meetings, assigned roles, and a Kanban board helped us stay organized and remove conflicts. We saw firsthand the value of clear communication, especially when collaborating remotely. We gained experience writing and running unit and integration tests. We learned how to set up and maintain a CI/CD pipeline using GitHub Actions and Netlify. We improved our ability to give and receive feedback through code reviews and testing each other's features. Most importantly, we learned how to work as a team, balancing technical work, planning, and collaboration to build a functional app under deadlines.

In the end, this project gave us valuable experience with the full software development lifecycle. Planning and coding to testing and deployment. Additionally, we learned the importance of communication and working together as a team.

Appendix

List of group members and Contributions:

Ash Aung:

- Basic Setup for all pages
- Header, Footer, Routing
- Built the Planning Page and its features
- Hosted the site on Netlify
- Managed the GitHub repository:
 - Setup initial CI/CD pipeline
 - Managed GitHub branching and pull request flow
- Wrote and revised documentation for final report
- Edited and captioned peer review video
- Captioned the final submission video

Amir:

- Implemented all Google Maps related features
- Managed all location-based functionality
- Managed the GitHub repository:
- Managed the GitHub repository:
 - Handled GitHub workflows and issues
 - Resolved merge conflicts
 - Set up the final CI/CD automation
 - Managed GitHub branching and pull request flow
- Fixed bugs and improved feature reliability
- Wrote and revised final report sections
- Created data diagrams

Damon:

- Started initial Google Maps integration
 - Adding a restaurant by right clicking on the map
- Developed Explore Page
 - Built features: “Explore Cuisines,” “Share a Restaurant,” “Start a Food Trend”
 - Integrated Firebase for user content
 - Created the video, header animations, and button animation for the page,
- Created and edited video presentations both for peer-review and final submission
- Designed Figma wireframes and prototype
- Wrote user personas

Jim:

- Developed AI Chatbot (Gemini API) and all of its features
- Created front page UI and animations
 - Created the animations on the front page
 - Typing animation
 - Slideshow animation
- Optimized site for mobile and tablet
- Wrote unit and integration tests
 - Wrote all the Unit and integration tests for each feature
 - Ensured seamless interaction between components, including API communication and user input handling
 - Wrote dedicated tests for Google Maps integration and Gemini chatbot logic
- Managed the GitHub repository:
 - Handled GitHub workflows and issues
 - Managed GitHub branching and pull request flow
- Contributed to documentation and final report

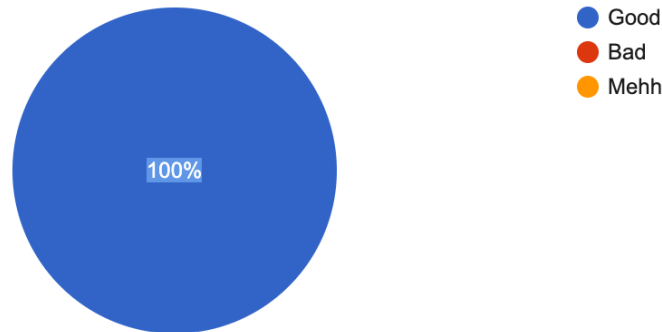
Peer Testing Feedback Form: <https://forms.gle/HuwL9TiMErQcmT9F8>

Survey Questions & Answers:

1. What are your overall thoughts on FoodieFind?
 - a. Good
 - b. Bad
 - c. Meh

What are your overall thoughts on FoodieFind?

4 responses



2. What did you like most about your experience?
 - a. It is interactive and most of the features work
 - b. Yall were really nice and explain your projects very well.
 - c. The ai search of the meal planning
 - d. Ui is pretty cool and its pretty user friendly
3. What aspects of the app felt confusing or frustrating?
 - a. Link redirection to a some places do not work
 - b. no, its pretty interesting going through the app
 - c. Nothing its very clean and easy to use
 - d. location was not correct sometimes
4. How easy/difficult was it to find and use the interactive map? Do you have any suggestions on how to improve it?
 - a. It is easy and works fine
 - b. i find it easy to use the map.
 - c. no it was perfect
5. Was the ChatBox feature intuitive? How was your experience with it?
 - a. The chat box can use more whitespace so that it looks good
 - b. ChatBox was a nice addition especialy for people who cant decide on what to eat and need some suggestions. the AI is perfect for that. IT can also help narrow down options for users.
 - c. it went around in loops 😞
6. How was the meal planning feature? Did you have any difficulty with it?
 - a. Was easy

- b. I have yet to test this feature :(
 - c. nope it was pretty good except the info can be a bit inaccurate and outdated
- 7. How'd you find the Explore page? Did you find any of the features on the page useful?
 - a. Real reviews is a great feature but was broken
 - b. I like it because if I feel like having a certain cuisine I can use that page instead of looking up and do research on the restaurant.
 - c. yeah it was pretty good
- 8. What features or improvements would you like to see in future updates?
 - a. The two broken features can be fixed to make it a great project
 - b. I don't have anything for now.
 - c. correct location in the explore section and the chatbox working right